

NEIGE & SOLEIL

Application Client Lourd — Java / Swing / MySQL

Documentation Technique

Juin 2026

1. Présentation du projet

Neige & Soleil est une application de gestion de résidences de vacances à la montagne. Elle est développée en Java avec une interface graphique Swing (client lourd) et communique avec une base de données MySQL locale.

L'application permet de gérer l'ensemble des opérations métier : propriétaires, appartements, clients, réservations, personnel, matériel de location et facturation.

1.1 Stack technique

Composant	Technologie
Langage	Java (JDK 8+)
Interface graphique	Java Swing (JFrame, JPanel, JTable...)
Base de données	MySQL 5.7+
Connecteur BDD	JDBC — pilote com.mysql.cj.jdbc.Driver
IDE recommandé	Eclipse
Architecture	MVC (Modèle – Vue – Contrôleur)

1.2 Structure du projet

Package / Dossier	Rôle
controleur/	Classes entités (POJO) + Contrôleur + Point d'entrée
modele/	Accès base de données (BDD.java + Modele.java)
vue/	Toutes les fenêtres et panels Swing
src/images/	Logo de l'application (logo.png)
src/sql/	Script SQL de création de la base (neige_soleil.sql)

2. Architecture MVC

Le projet suit rigoureusement le patron de conception MVC :

2.1 Schéma des couches

- Vue (package vue) : fenêtres JFrame et panels JPanel. Aucune logique métier.
- Contrôleur (package controleur) : reçoit les actions de la vue, appelle le modèle, renvoie les données.
- Modèle (package modele) : exécute les requêtes SQL via JDBC et retourne des objets Java.

2.2 Flux d'une action utilisateur

Exemple — connexion de l'utilisateur :

- L'utilisateur saisit email + mot de passe dans VueConnexion et clique sur Valider.
- VueConnexion appelle Controleur.selectWhereUser(email, mdp).
- Controleur délègue à Modele.selectWhereUser(email, mdp).
- Modele exécute `SELECT * FROM user WHERE email=... AND mdp=...` via JDBC.
- Si un User est trouvé, Neige_soleil.creerDetruireVueGenerale(true, unUser) ouvre la fenêtre principale.

3. Point d'entrée — Neige_soleil.java

La classe Neige_soleil (package controleur) contient la méthode main et joue le rôle de chef d'orchestre entre les vues.

3.1 Méthodes publiques

Méthode	Description
main(String[] args)	Crée et affiche la VueConnexion au démarrage.
rendreVisibleVueConnexion(boolean action)	Affiche ou masque la fenêtre de connexion.
creerDetruireVueGenerale(boolean action, User unUser)	Crée la VueGenerale (action=true) ou la détruit (action=false).

4. Couche Modèle

4.1 BDD.java — Connexion à la base

La classe BDD (package modele) gère exclusivement la connexion / déconnexion JDBC.

Méthode	Rôle
chargerPilote()	Charge le driver MySQL : com.mysql.cj.jdbc.Driver
seConnecter()	Ouvre la connexion JDBC vers jdbc:mysql://localhost/neige_soleil
SeDeConnecter()	Ferme proprement la connexion.
getMaConnection()	Retourne l'objet Connection actif.

Configuration par défaut :

- Serveur : localhost
- Base : neige_soleil
- Utilisateur : root
- Mot de passe : (vide)

4.2 Modele.java — Requêtes SQL

La classe Modele contient toutes les méthodes d'accès aux données. Elle utilise une instance statique de BDD.

Méthode utilitaire

Méthode	Description
executerRequete1(String requete)	Exécute tout INSERT, UPDATE ou DELETE. Gère la connexion et la fermeture automatiquement.

Opérations disponibles par entité

Entité	Insert	Select (avec filtre)	Update	Delete
User	—	selectWhereUser(email, mdp)	updateUserProfil(user)	—
Proprietaire	insertProprietaire	selectAllProprietaires(filtre)	updateProprietaire	deleteProprietaire
Appartement	insertAppartement	SelectAllAppartements(filtre)	updateAppartement	deleteAppartement
Client	insertClient	selectAllClients(filtre)	updateClient	deleteClient
Reservation	insertReservation	selectAllReservations(filtre)	updateReservation	deleteReservation
Materiel	insertMateriel	SelectAllMateriels(filtre)	updateMateriel	deleteMateriel
Personnel	insertPersonnel	selectAllPersonnel(filtre)	updatePersonnel	deletePersonnel

Facture	—	recupererDonneesFacture(id_reser)	—	—
---------	---	-----------------------------------	---	---

5. Couche Contrôleur

5.1 Controleur.java

Classe statique faisant le pont entre la Vue et le Modèle. Chaque méthode est une simple délégation vers Modele.

Exemple de méthode :

```
public static ArrayList<Appartement> selectAllAppartements(String filtre) {
    return Modele.SelectAllAppartements(filtre);
}
```

5.2 Classes entités (POJO)

Chaque entité métier dispose de sa propre classe Java avec constructeurs, getters et setters.

Classe	Attributs principaux
User	id_perso, nom, prenom, email, mdp, tel, role
Proprietaire	idProprio, nom, prenom, adresse, email, tel, iban
Appartement	id_appart, num_appart, type_appart, surface, capacite_accueil, exposition, distance_pistes, prix_hebdo, image, idproprio
Client	idclient, nom, prenom, email, tel
Reservation	id_reser, date_debut_loc, date_fin_loc, nb_personnes, id_client, id_appart, id_employe
Personnel	id_employe, nom, prenom, email, tel, role
Materiel	id_mat, libelle_mat, type_mat, etat, prix_jour, idproprio
Tableau	donnees (Object[][]), entetes (String[]) — extends AbstractTableModel

Chaque classe dispose de deux constructeurs :

- Constructeur complet avec l'id (pour les données lues depuis la BDD).
- Constructeur sans id (pour les insertions, l'auto-increment MySQL gère l'id).

6. Couche Vue (Interface graphique)

6.1 VueConnexion — Fenêtre de connexion

Fenêtre JFrame affichée au démarrage. Dimensions : 300 x 500 px.

Élément	Description
Logo	Affiché depuis src/images/logo.png (250x200 px)
txtEmail (JTextField)	Champ de saisie de l'email
txtMdp (JPasswordField)	Champ mot de passe masqué
btValider	Déclenche la vérification des identifiants via Controleur
btAnnuler	Vide les deux champs
Touche Entrée	Équivalent au clic sur Valider (KeyListener sur les champs)

6.2 VueGenerale — Fenêtre principale

Fenêtre JFrame principale. Dimensions : 1100 x 600 px. Reçoit l'objet User connecté en paramètre.

Barre de navigation (GridLayout 1x9) :

Bouton	Panel affiché
Profil	Panelprofil (infos de l'utilisateur connecté)
Propriétaire	PanelProprietaire
Clients	PanelClient
Appartements	PanelAppartements
Réservation	PanelReservation
Matériel	PanelMateriels
Factures	PanelFacture
Personnel	PanelPersonnel
Quitter	Ferme la vue, rouvre la connexion

6.3 PanelPrincipal — Classe abstraite de base

Tous les panels héritent de PanelPrincipal (extends JPanel, abstract). Il définit les dimensions communes (960x500 px), la couleur de fond grise et affiche un titre centré en Arial gras 18pt.

6.4 Tableau.java — Modèle de table générique

La classe `Tableau` (extends `AbstractTableModel`) est utilisée par tous les panels pour alimenter les `JTable`. Elle prend une matrice `Object[][]` et un tableau `String[]` d'entêtes.

Méthode	Description
<code>getRowCount()</code>	Retourne le nombre de lignes
<code>getColumnCount()</code>	Retourne le nombre de colonnes
<code>getValueAt(i, j)</code>	Retourne la cellule <code>[i][j]</code>
<code>getColumnName(i)</code>	Retourne le nom de la colonne <code>i</code>
<code>setDonnees(Object[][])</code>	Met à jour les données et notifie la <code>JTable</code> (<code>fireTableDataChanged</code>)

7. Base de données — neige_soleil

7.1 Tables

Table	Clé primaire	Description
User	id_perso (AUTO_INCREMENT)	Table centrale partagée par Client et Propriétaire (héritage)
PROPRIETAIRE	id_proprio (INT)	Propriétaires des appartements
APPARTEMENT	id_appart (AUTO_INCREMENT)	Appartements à louer, liés à un propriétaire
CLIENT	id_client (INT)	Clients locataires (synchronisé avec User via trigger)
RESERVATION	id_reser (AUTO_INCREMENT)	Réservations liant client, appartement et employé
PERSONNEL	id_employe (AUTO_INCREMENT)	Employés gérant les réservations
MATERIEL	id_mat (AUTO_INCREMENT)	Matériel de sport disponible à la location
CONTRAT	id_contrat (AUTO_INCREMENT)	Contrats associés aux appartements
EQUIPEMENT	id_equip (AUTO_INCREMENT)	Équipements des appartements
PAIEMENT	id_paiement (AUTO_INCREMENT)	Paiements liés aux réservations (1-1)
DISPOSER	(id_appart, id_equip)	Table de liaison Appartement — Équipement
LOUER_MATERIEL	id_loc_mat (AUTO_INCREMENT)	Location de matériel par un client

7.2 Triggers

La base utilise un mécanisme d'héritage simulé via des triggers MySQL pour maintenir la table User cohérente avec CLIENT et PROPRIETAIRE.

Trigger	Événement	Effet
insert_client	BEFORE INSERT sur CLIENT	Calcule un id_client unique, insère dans User avec role='client'
update_client	BEFORE UPDATE sur CLIENT	Met à jour les infos dans User
delete_client	BEFORE DELETE sur CLIENT	Supprime l'entrée correspondante dans User
insert_proprio	BEFORE INSERT sur PROPRIETAIRE	Insère dans User avec role='proprietaire'

update_proprio	BEFORE UPDATE sur PROPRIETAIRE	Met à jour dans User
delete_proprio	BEFORE DELETE sur PROPRIETAIRE	Supprime dans User

7.3 Requête Facture

La génération de facture repose sur une requête SQL de jointure :

```
SELECT C.nom, C.prenom, A.type_appart, A.prix_hebdo,
       DATEDIFF(R.date_fin_loc, R.date_debut_loc) AS duree
FROM RESERVATION R
JOIN CLIENT C ON R.id_client = C.id_client
JOIN APPARTEMENT A ON R.id_appart = A.id_appart
WHERE R.id_reser = ?
```

8. Installation et lancement

8.1 Prérequis

- JDK 8 ou supérieur installé
- MySQL Server en fonctionnement sur localhost
- Pilote JDBC MySQL (mysql-connector-java) dans le classpath Eclipse

8.2 Création de la base de données

Exécuter le script SQL fourni dans src/sql/neige_soleil.sql dans un client MySQL :

```
mysql -u root -p < src/sql/neige_soleil.sql
```

Le script crée la base neige_soleil, toutes les tables, les triggers, et insère 10 appartements de démonstration.

8.3 Configuration de la connexion

La connexion est codée en dur dans Modele.java :

```
private static BDD uneBdd = new BDD("root", "", "neige_soleil",  
    "localhost");
```

Pour modifier les paramètres, adapter les arguments : utilisateur, mot de passe, nom de base, adresse du serveur.

8.4 Lancement

Depuis Eclipse, exécuter la classe controleur.Neige_soleil (méthode main). La fenêtre de connexion s'affiche.

Identifiants par défaut (à créer directement en base) :

- Email : jean@mail.com
- Mot de passe : 123

9. Points d'amélioration identifiés

Point	Problème actuel	Recommandation
Injection SQL	Les requêtes sont construites par concaténation de chaînes	Utiliser des PreparedStatement
Mots de passe	Stockés en clair dans la base (mdp VARCHAR(50))	Hasher avec BCrypt ou SHA-256
Connexion BDD	Ouverte et fermée à chaque requête	Utiliser un pool de connexions (ex: HikariCP)
Gestion d'erreurs	Affichage console uniquement (System.out.println)	Utiliser un logger (java.util.logging ou SLF4J)
User.getRole()	Retourne null au lieu de this.role	Corriger le getter pour retourner this.role
User.getId_perso()	Retourne null (String) au lieu de int	Corriger ou supprimer cette méthode
filtre Reservation	selectAllReservations ignore le paramètre filtre	Appliquer le filtre dans la requête SQL
Materiel insert	L'id_proprio n'est pas inséré en base	Corriger la requête INSERT INTO materiel

10. Résumé des fichiers source

Fichier	Package	Rôle
Neige_soleil.java	controleur	Classe main — chef d'orchestre
Controleur.java	controleur	Pont Vue ↔ Modèle (méthodes statiques)
User.java	controleur	Entité utilisateur connecté
Proprietaire.java	controleur	Entité propriétaire d'appartement
Appartement.java	controleur	Entité appartement
Client.java	controleur	Entité client locataire
Reservation.java	controleur	Entité réservation
Personnel.java	controleur	Entité employé
Materiel.java	controleur	Entité matériel de location
Tableau.java	controleur	Modèle générique pour JTable
BDD.java	modele	Connexion / déconnexion JDBC
Modele.java	modele	Toutes les requêtes SQL
VueConnexion.java	vue	Fenêtre de connexion
VueGenerale.java	vue	Fenêtre principale avec navigation
PanelPrincipal.java	vue	Classe abstraite de base pour tous les panels
Panelprofil.java	vue	Panel profil de l'utilisateur connecté
PanelProprietaire.java	vue	CRUD Propriétaires
PanelAppartements.java	vue	CRUD Appartements
PanelClient.java	vue	CRUD Clients
PanelReservation.java	vue	CRUD Réservations
PanelMateriels.java	vue	CRUD Matériel
PanelPersonnel.java	vue	CRUD Personnel
PanelFacture.java	vue	Génération de factures
PanelPrincipal.java	vue	Panel vide de bienvenue
neige_soleil.sql	sql	Script de création BDD + triggers + données